DAA Project: AI-Based Complexity Prediction Tool

Project Title: AI-Based Complexity Prediction Tool — Implementation using Python

1. Project Abstract

The AI-Based Complexity Prediction Tool is designed to analyze Python source code and predict its likely time complexity, such as \(O(1)\), \(O(n)\), \(O(n^2)\), or \(O(\log n)\).

The tool examines code features, including loops, nested loops, recursion, and sorting operations. These features are passed to a machine-learning model that predicts the most likely complexity class.

The project helps students understand algorithm efficiency and compare the theoretical complexity of different algorithms. The prediction is an estimate and should be verified using algorithm analysis and runtime testing.

2. Objectives

- Analyze Python code automatically.
- Identify loops, nested loops, and recursion.
- Predict algorithmic time complexity using machine learning.
- Compare predicted complexity with theoretical complexity.
- Display results through a simple user interface.

3. Technologies Required

Component	Technology
Programming language	python
Machine	scikit-learn
Code analysis	python AST
Data processing	pandas
User interface	Streamlit
Graphs	Matplotlib

4. System Architecture Diagram

1. User Input
Enter or paste Python source code

2. Code Preprocessing
Validate source code and parse it using Python AST

3. Feature Extraction
Detect loops, nested loops, recursion, and sorting operations

4. Machine Learning Model
Decision Tree classifier trained on labeled algorithm examples

5. Complexity Prediction
Predict a likely class: O(1), O(log n), O(n), O(n log n), O(n^2)

6. Result Display
Streamlit interface displays predicted complexity and extracted features

. Step-by-Step Implementation
Step 1: Install the required libraries
Open your terminal and run:

</> Bash
pip install scikit-learn streamlit pandas

Step 2: Create the project folder

</> Plain text

```
AI-Complexity-Predictor/
├── app.py
├── train_model.py
├── dataset.csv
└── README.md
```

Step 3: Create the training dataset
Create dataset.csv with the following sample data:

</> CSV

```
loops,nested_loops,recursion,sorting,complexity
0,0,0,0,O(1)
1,0,0,0,O(n)
2,1,0,0,O(n^2)
3,1,0,0,O(n^2)
1,0,1,0,O(log n)
1,0,0,1,O(n log n)
```

These rows are illustrative examples, not a sufficient dataset for a reliable model. In the full project, create a larger, correctly labeled dataset using known algorithms and their theoretical complexities.

Step 4: Train the machine-learning model
Create train_model.py:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pickle

data = pd.read_csv("dataset.csv")

X = data[
   ["loops", "nested_loops", "recursion", "sorting"]
]
y = data["complexity"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Model accuracy:", accuracy_score(y_test, predictions))

with open("model.pkl", "wb") as file:
   pickle.dump(model, file)

print("Model saved successfully.")
```

out put :

```
python train_model.py
```

Step 5: Create the prediction tool
Create app.py:

```
import streamlit as st
import pickle
import ast

st.title("AI-Based Complexity Prediction Tool")
st.write("Predict the likely time complexity of Python code.")
```

```
with open("model.pkl", "rb") as file:
    model = pickle.load(file)

code = st.text_area(
    "Enter Python code:",
    height=200,
    placeholder="for i in range(n):\n    print(i)"
)

def extract_features(source):
    tree = ast.parse(source)

    loops = 0
    nested_loops = 0
    recursion = 0
    sorting = 0

    def visit(node, depth=0):
        nonlocal loops, nested_loops, recursion, sorting

        if isinstance(node, (ast.For, ast.While)):
            loops += 1
            if depth > 0:
                nested_loops += 1
            depth += 1

        if isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name):
                if node.func.id in ("sort", "sorted"):
                    sorting += 1
            elif isinstance(node.func, ast.Attribute):
                if node.func.attr == "sort":
                    sorting += 1

        if isinstance(node, ast.FunctionDef):
            for child in ast.walk(node):
                if (
                    isinstance(child, ast.Call)
                    and isinstance(child.func, ast.Name)
                    and child.func.id == node.name
                ):
                    recursion += 1
                    break

        for child in ast.iter_child_nodes(node):
            visit(child, depth)

    visit(tree)
    return [[loops, nested_loops, recursion, sorting]]

if st.button("Predict Complexity"):
    try:
        features = extract_features(code)
        result = model.predict(features)[0]
```

```
    st.success(f"Predicted Time Complexity: {result}")
    st.caption(
        "This is an estimate based on simple code features, "
        "not a proof of the algorithm's actual complexity."
    )
except SyntaxError:
    st.error("Invalid Python syntax. Please check your code.")
except Exception as error:
    st.error(f"Prediction failed: {error}")
```

Step 6: Run the application

Open your terminal in the project folder:

Out put:

streamlit run app.py

The tool will open in your browser. Enter Python code and click Predict Complexity.

6. Expected Output

Example input:

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Expected theoretical complexity: \(O(n^2)\).

The simple sample model may predict a different result because its features and training dataset are limited. Improving feature extraction and training on a larger, representative dataset is an important part of the project.

7. DAA Concepts Used

- Asymptotic notation: Big-O.
- Best-case, average-case and worst-case analysis.
- Time complexity and space complexity.
- Recursion analysis.
- Loop analysis.
- Algorithm classification.
- Machine-learning-based prediction.

8. Future Enhancements

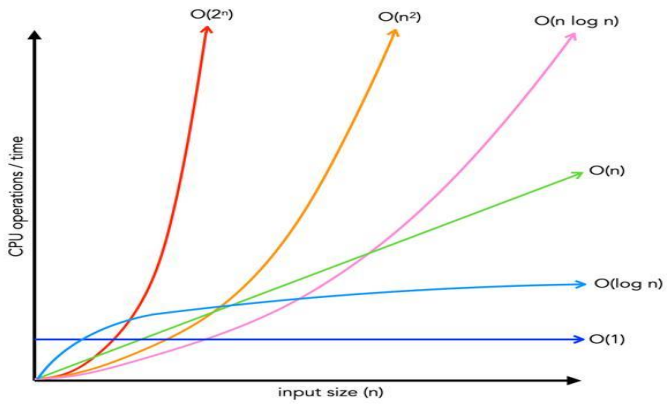
- Support multiple programming languages.
- Estimate space complexity.
- Analyze recursive algorithms more accurately.
- Compare predicted and measured runtime.
- Add graphs showing execution time for different input sizes.
- Build a larger dataset of standard algorithms.

Important: This starter implementation is suitable for learning and demonstrating the concept, but it is not yet a dependable AI complexity analyzer. A strong DAA submission should include a better dataset, AST-based loop-depth analysis, recursion detection, runtime experiments, and a comparison against known theoretical complexities.

3. Technologies used

- Python — core programming language
- AST — source-code structure analysis
- Pandas — dataset processing
- Scikit-learn — Machine Learning model
- Streamlit — web-based user interface
- Matplotlib — optional charts and visualizations

Important limitation: This architecture predicts complexity from code features; it does not guarantee the true Big-O complexity. A stronger version should account for input-size behavior, loop bounds, recursion, and the algorithm's logic.



This graph compares how the number of operations grows as the input size increases.

- \(O(1)\) — Constant
- \(O(\log n)\) — Logarithmic
- \(O(n)\) — Linear
- \(O(n\log n)\) — Linearithmic
- \(O(n^2)\) — Quadratic

6. Machine Learning workflow

1. Collect training data: Prepare example algorithms and their manually verified complexity labels.
2. Extract features: Analyze each example with the AST parser.
3. Prepare the dataset: Convert the features and labels into a structured CSV or DataFrame.
4. Train the model: Use a Decision Tree or another classifier to learn patterns.
5. Evaluate performance: Test on examples not used during training and report accuracy and per-class metrics.
6. Deploy the model: Load the trained model in a Streamlit interface.

The initial dataset should contain many varied examples for each complexity class. A model trained on just a few rows is only a demonstration, not a dependable predictor.

9. Advantages and limitations

Advantages

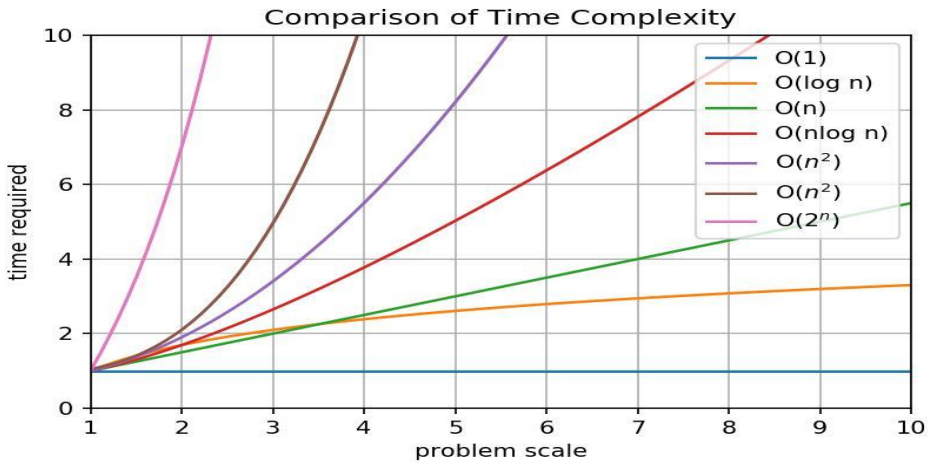
- Helps students learn Big-O notation.
- Automates some repetitive code-analysis tasks.
- Combines DAA concepts with Machine Learning.
- Can provide an interactive demonstration through a web interface.
- Can be extended with visualizations and execution-time measurements.

Limitations

- Code structure does not always reveal true complexity.

- Dynamic behavior and input-dependent branches can be difficult to analyze statically.
- Small or biased datasets can cause incorrect predictions.
- Runtime measurements can vary by hardware, interpreter, and input data.

A stronger version can combine static analysis, theoretical algorithm rules, controlled benchmarking, and ML predictions.



This graph helps explain why some algorithms become much slower as the input size increases.

10. Implementation roadmap

Phase	Work to complete
Phase 1	Create the project folder and Python environment
Phase 2	Implement AST parsing and feature extraction
Phase 3	Prepare a labeled dataset
Phase 4	Train and evaluate the ML classifier
Phase 5	Build the Streamlit interface
Phase 6	Test with known algorithms and fix incorrect predictions
Phase 7	Prepare the report, architecture diagram, screenshots, and presentation

11. How to make the project stronger

For a college-level DAA project, I recommend a hybrid approach:

- First, use static analysis to identify recognizable algorithm patterns.
- Next, apply rule-based complexity estimation for straightforward loops and recursion.
- Then, use the ML model to classify patterns that are harder to identify.
- Finally, optionally compare theoretical estimates with measured runtime for several input sizes.

This makes the project more technically meaningful than simply training a model on loop counts.